

Trabajo Práctico — Metodologías Ágiles para el Desarrollo de Software

Consigna

Datos de la cursada:

- Modalidad: **en grupos** de **hasta 4 integrantes**.
- Fecha de entrega: **semana del 12 de octubre de 2026**, en el repositorio del grupo.
- Defensa: **oral**, con fecha **a acordar con cada equipo** después de la entrega.

1. Objetivo

Construir una **aplicación web funcional** aplicando, de punta a punta, las prácticas técnicas del desarrollo ágil: **TDD** para la lógica, **ATDD** para el comportamiento de la app, y un **pipeline de Integración Continua y Despliegue Continuo (CI/CD)** que compile, pruebe, mida calidad y despliegue a producción de forma automática.

No se evalúa solo que la app funcione: se evalúa que el **proceso** (test primero, separación de responsabilidades, pipeline automatizado, trabajo de equipo) se haya seguido y que el grupo pueda **explicarlo**.

2. La aplicación

La aplicación a construir es el **Ahorcado web**, con la **lógica de juego separada de la interfaz**, siguiendo el método de la **Guía ATDD** (`GUIA-ATDD-IA-Ahorcado.md`): doble loop outside-in (AT sobre la UI + Unit Tests sobre el dominio), un AT a la vez, rotación de autor y commits rojo/verde.

La Guía ATDD define **cómo desarrollar**. Esta consigna define **qué entregar** y el **pipeline** que lo respalda.

3. Prácticas y requisitos técnicos

Cada requisito incluye qué se pide y **cómo se verifica** (criterio de aprobación).

3.1 Unit Tests usando TDD

- **Qué se pide:** la **lógica de dominio** (el objeto/módulo del ahorcado) se construye con TDD: por cada porción de lógica, primero el test que falla (rojo), luego el código mínimo (verde), luego refactor.
- **Verificación:** el historial de git muestra commits **rojo** → **verde** por cada test (el rojo como ancestro del verde), con prefijos **RED:** / **GREEN:** en el mensaje (ver §4). La cobertura del dominio tiende a ~100% (ver 3.6). La lógica **no** depende del DOM ni de la UI.

3.2 Desarrollo de la UI web con Acceptance Tests (ATDD)

- **Qué se pide:** la funcionalidad visible se maneja desde **Acceptance Tests** que actúan como un usuario sobre la **app real corriendo** (no sobre un componente en memoria). Cada feature nace de un AT en rojo.
- **Mínimo: 4 Acceptance Tests.**
- **Verificación:** existen al menos 4 AT automatizados, verdes, que ejercitan la app de punta a punta (UI + dominio).

3.3 Framework de Acceptance Tests: Cucumber (o equivalente por lenguaje)

- **Qué se pide:** los AT se escriben en estilo **Cucumber/Gherkin** (**Given** / **When** / **Then** en archivos `.feature` + *step definitions*). Equivalentes por lenguaje: **Reqnroll** (C#), **Cucumber-JVM** (Java/Kotlin), **behave** o **pytest-bdd** (Python), **CucumberJS** (JS/TS), etc.
- **Automatización del navegador:** **Selenium** u *otro framework similar* (Playwright, Cypress, WebDriver) para manejar la UI real.
- **Verificación:** hay archivos `.feature` legibles por negocio y sus *steps* automatizan la app en un navegador.

3.3.1 Trazabilidad con las Historias de Usuario

- **Qué se pide:** los Acceptance Tests **salen de las Historias de Usuario** del grupo (su Story Map): cada AT confirma un criterio de aceptación de una historia. La cadena **Historia** → **criterio** → **escenario (AT)** → **Unit Tests** tiene que poder recorrerse **en los dos sentidos** mirando el repo.
- **El mecanismo es libre y es parte del trabajo explorarlo:** una **Característica** por historia, etiquetas de Gherkin (`@HU-02`), issues de un tablero referenciados desde los commits, una matriz en `NOTES.md`, o una combinación (ver Guía ATDD §4.1). El grupo elige y **justifica** su elección en la defensa.
- **Verificación:**
 1. Cada historia tiene un **identificador estable** y el Story Map / tablero está **accesible** desde el repo (link en el README o en `NOTES.md`).
 2. **Cada AT referencia su historia.** Un escenario sin historia no cumple.
 3. Dada **una historia**, el grupo muestra desde el repo sus escenarios, sus UTs y si está terminada (todos sus escenarios verdes). Dado **un test**, muestra de qué historia sale.

3.4 Single Repository

- **Qué se pide:** un **único repositorio** contiene todo el trabajo: lógica, UI, unit tests, acceptance tests y la configuración del pipeline.

- **Verificación:** un solo repo, con historial que evidencia el proceso y la **rotación de autores** entre los integrantes (`git shortlog -sne`).

3.5 CI Server — el pipeline

Se debe configurar un **servidor de CI** —todos con tier gratuito— como **GitHub Actions** (gratis en repos públicos), **GitLab CI/CD**, **CircleCI** o **Azure Pipelines**, que ejecute, de forma automática ante cada push, un pipeline con las siguientes **etapas en orden**. Una etapa que falla **corta** el pipeline (no se llega a desplegar algo roto).

#	Etapas	Qué hace	Herramientas sugeridas
1	Compila / Build	El proyecto compila/builda sin errores	la del stack elegido
2	Unit Tests	Corre los unit tests del dominio; rojo si alguno falla	Vitest, xUnit, JUnit, pytest...
3	Code Coverage	Mide, publica y corta si baja del umbral	coverage del runner / Sonar
4	Análisis Estático	Analiza calidad del código y aplica un <i>quality gate</i> que corta	SonarQube Cloud (ex-SonarCloud, gratis para OSS) o alternativas: CodeQL (gratis en repos públicos), Codacy, linters (ESLint...)
5	Acceptance Tests	Levanta la app y corre los AT (Cucumber + Selenium/...)	el framework de 3.3
6	Deploy a Producción	Despliega a un servidor accesible por Internet	ver 3.7

Orden de AT y deploy: lo recomendado es correr los AT **antes** del deploy (como compuerta: si los AT fallan, no se despliega). Si el grupo prefiere correr AT **contra producción** como *smoke test* posterior al deploy, debe justificarlo.

3.5.1 Compila

El pipeline builda el proyecto desde cero. **Criterio:** build verde.

3.5.2 Ejecuta los Unit Tests

El pipeline corre **todos** los unit tests. **Criterio:** 100% en verde; si falla uno, el pipeline falla.

3.6 Muestra resultados de Code Coverage

El pipeline **mide, publica y controla** la cobertura (resumen visible en el run y/o en el dashboard de calidad). **Criterio:** hay un **umbral del 90% configurado sobre el dominio** (`src/domain/` o su equivalente en el stack elegido) y, si la cobertura queda por debajo, **el job falla** (exit $\neq$ 0) y **corta el pipeline**. El umbral vive en la configuración del runner, versionada en el repo (`coverage.thresholds` en Vitest, `--cov-fail-under` en pytest, `/p:Threshold` en coverlet, etc.) — no alcanza con mirarlo a ojo en el reporte.

El umbral se aplica **sobre el dominio**, no sobre todo el proyecto: la UI se cubre con los Acceptance Tests, no con unit tests. Y recordar: *cobertura alta ≠ cero bugs* (ver Guía ATDD).

3.7 Ejecuta Análisis Estático de Código

Integrar **SonarQube Cloud** (antes *SonarCloud*; su plan **SonarQube for OSS** es gratis para repos públicos) o alternativa (**CodeQL**, gratis en repos públicos de GitHub; Codacy; linters), con un **quality gate**.

Criterio (se verifica en el YAML del pipeline y en la corrida):

1. El análisis corre en el pipeline **en cada push**.
2. El **quality gate hace fallar el job**: si el gate no pasa, el step termina con `exit ≠ 0` y el pipeline se corta. En la práctica esto significa esperar el resultado del gate y romper con él (p. ej. `sonarqube-quality-gate-action` con `continue-on-error` **ausente o en false**).
3. **No se acepta `continue-on-error: true`** ni ninguna variante que deje el job en verde con el gate en rojo.
4. **Compilar no es análisis estático**. `tsc`, `javac`, `dotnet build` o cualquier *typecheck* pertenecen a la etapa 1 (Build) y **no cuentan** como análisis estático. Tampoco cuenta un linter que corre en modo advertencia sin romper.

Estos cuatro puntos son la causa de la mayoría de los rechazos del cuatrimestre pasado. Si el pipeline está verde con el gate en rojo, la etapa **no está cumplida**, por más que el step aparezca en el YAML.

3.8 Deploy a Producción

El pipeline despliega automáticamente a un **servidor en Internet** accesible por una URL pública. Como el ahorcado es **100% del lado del cliente**, alcanza con un **hosting estático gratuito**: **GitHub Pages**, **Netlify**, **Vercel** o **Cloudflare Pages**. Si el grupo elige una arquitectura con backend, opciones con tier gratuito: **Render**, **Google Cloud Run** o **Azure App Service (F1)**. (Atención: *Railway* y *Fly.io* ya no tienen plan gratuito, solo prueba con crédito.) **Criterio:**

1. La app está **online y jugable** en la URL entregada: lo que sirve esa URL es **el juego**, no un reporte de cobertura, un README renderizado ni una página por defecto.
2. El deploy lo hace **el pipeline**, no una mano: no vale subir el `dist/` a Netlify por la web.
3. El deploy está **encadenado** a las etapas anteriores (`needs:` en GitHub Actions, o `workflow_run` con `conclusion == 'success'`): si falla build, unit tests, coverage, análisis estático o AT, **no se despliega**. Un job de deploy que corre en paralelo o siempre no cumple.

3.9 Ejecuta los Acceptance Tests

El pipeline corre los **AT automatizados** (Cucumber/Gherkin + Selenium o similar). **Criterio:** los ≥ 4 AT corren en el pipeline y están en verde.

4. Trabajo de equipo (proceso)

- **Rotación de autor:** cada test lo arranca un integrante distinto, por turnos, commiteando con su **propia identidad**. **Piso verificable:** cada integrante arranca al menos 3 tests, es decir, es el autor de al menos 3 commits **RED:** (y del **GREEN:** correspondiente). Se verifica así:

```
git log --format='%an %s' | grep RED: | sort | uniq -c # cuántos RED: arrancó cada uno
```

Un integrante que no llega a ese piso **no aprueba el TP**, aunque el grupo entregue todo lo demás. Configurar bien `user.name` / `user.email` de cada uno desde el primer día: si commitean todos desde la misma máquina sin cambiar la identidad, el historial no lo puede demostrar.

- **Orden test-primero:** rojo antes que verde en el historial. **Integración solo en verde:** la rama principal (lo que evalúa CI) nunca queda rota.
- **Mensajes de commit:** prefijo **RED:** en el commit del test que falla y **GREEN:** en el que lo hace pasar (opcional **REFACTOR:**). Así el orden test-primero y la rotación se leen de un vistazo con `git log --oneline`.
- **Detalle del método:** ver Guía ATDD (§3 Reglas del proceso, §7 Evaluación).

5. Entregables

1. **URL del repositorio** (único), con el historial completo.
2. **URL de la app en producción** (online y jugable).
3. **Pipeline de CI/CD configurado y en verde** — link a la última corrida exitosa mostrando todas las etapas (build, unit tests, coverage, análisis estático, AT, deploy).
4. **Dashboard de análisis estático / cobertura** (p. ej. SonarQube Cloud) accesible.
5. **Los ≥ 4 archivos de Acceptance Test** (`.feature`) y sus *steps*.
6. El **NOTES.md** con la lista de unit tests que justifica cada Acceptance Test (ver Guía ATDD §5, paso 2 y §7).
7. El **link al Story Map / tablero** con las Historias de Usuario identificadas, y la **trazabilidad** Historia → AT → UT resuelta con el mecanismo elegido (§3.3.1).

6. Condiciones de aprobación

Para aprobar el TP, **todos** estos puntos deben cumplirse y poder defenderse:

- ☐ Lógica desarrollada con **TDD** (rojo→verde en el historial) y separada de la UI.
- ☐ **≥ 4 Acceptance Tests** en estilo Cucumber/Gherkin, automatizados y verdes.
- ☐ **Cada AT traza a una Historia de Usuario** del grupo, y cada historia tomada muestra sus AT y UTs desde el repo (§3.3.1).
- ☐ **Single repository** con rotación de autores visible: **cada integrante arranca ≥ 3 tests** (≥ 3 commits **RED:** propios).

- ☐ Pipeline de CI que **compila**.
- ☐ Pipeline que **ejecuta los unit tests**.
- ☐ Pipeline que **publica code coverage** y **corta si el dominio baja del 90%**.
- ☐ Pipeline que ejecuta **análisis estático** con un quality gate **que corta el pipeline** (y que **no** es compilar/typecheck).
- ☐ Pipeline que **ejecuta los AT**.
- ☐ Pipeline que **deploya a producción, encadenado** a las etapas anteriores; la URL sirve **el juego** y está **online**.
- ☐ **Defensa oral:** el grupo explica cualquier test/decisión/etapa del pipeline.

Aprobación Directa: además de lo anterior, el grupo agrega **al menos 4 funcionalidades** extra aplicando el mismo proceso (ver Guía ATDD §9: cada feature con su historia, su AT + unit tests y la misma trazabilidad, sin guía paso a paso).

7. Fechas y entrega

- **Entrega:** durante la **semana del 12 de octubre de 2026**, en el **repositorio del grupo**. No hay formulario ni entrega por campus: lo que está en el repo al cierre de esa semana es lo que se corrige. El estado del repo queda registrado en el historial de git, así que no hace falta "avisar" que entregaron — pero sí que el repo sea **público/accesible** y que la URL de producción esté online.
- **Defensa oral:** a partir de esa fecha, con día y horario **a acordar con cada equipo**. Se defiende en grupo y **cada integrante** tiene que poder explicar cualquier test, decisión o etapa del pipeline (ver §6 y Guía ATDD §7).